UNIT IV Advanced Image Processing Using Open CV

Image Blending

Image blending is a fundamental technique in advanced image processing where two images are combined into a single composite image. The key objective is to create a seamless transition between images—often to fuse features, overlay images, or create artistic effects—without harsh boundaries or visual disruption.

Key Ideas Behind Blending Two Images

1. **Pixel-wise Combination:**
    - Conceptually, each pixel in the output image is computed as a blend of corresponding pixels from the two input images.
    - The blending is generally a weighted sum:

$$I_{output}(x,y) = \alpha \times I1(x,y) + (1-\alpha) \times I2(x,y)$$

where $\alpha\alpha$ (alpha) is the blending factor controlling the contribution of each image at pixel $(x,y)(x,y)$.

2. **Alpha Blending:**
    - When you want a smooth mix, alpha blending applies weights between 0 and 1.
    - $\alpha = 0.5\alpha = 0.5$ means an equal mix; closer to 0 means mostly second image; closer to 1 means mostly first image.
    - This concept is also foundational in computer graphics transparency and compositing.
3. **Region-based or Mask-based Blending:**
    - Sometimes blending is selectively applied over certain regions.
    - Masks (grayscale or binary images) define where and how strongly one image appears over the other.
    - Masks allow complex blending patterns, providing artistic control or correcting image alignments.
4. **Seamless or Multi-band Blending:**

- Simple linear blending can create ghosting or visible seams when images have distinct features or misalignments.
- Techniques like **pyramid blending (multi-band blending)** decompose images into multiple spatial frequency bands.
- Each band is blended separately before recombining, ensuring smooth transitions across edges and avoiding artifacts.
- This approach respects different scales of detail, blending large regions gently while preserving sharp details where appropriate.

5. **Color Space Considerations:**
    - Blending can be done in RGB, but sometimes better results appear when working in other color spaces (e.g., HSV or LAB).
    - This can help avoid color distortion when blending or preserve luminance and chrominance components distinctly.
6. **Geometric Alignment:**
    - For blending to work meaningfully, the images must be spatially aligned (registered).
    - Misalignments cause visible blur, double edges, or ghost images.
    - Image alignment prior to blending can involve feature matching, homography estimation, or warping.

Why Use OpenCV for Image Blending Conceptually?

- OpenCV provides efficient **image representations** and **matrix operations** allowing pixel-wise arithmetic easily.
- It supports blending functions and pyramid representations natively for multi-band blending.
- Through OpenCV, concepts like masks, alpha channels, and image pyramids are gym for experimenting blending effects.

Summary of the Key Concept:

- **Blending two images involves controlled, pixel-level weighted summation facilitated by an alpha coefficient or mask.**
- **Advanced blending techniques like pyramid blending perform blending at multiple detail levels to avoid artifacts.**

- **Proper alignment and sometimes color space choice are preconditions for high-quality blending.**

This conceptual framework enables understanding beyond code, focusing on the principles governing how images fuse visually pleasingly.

Changing Contrast and Brightness

Changing contrast and brightness in images is a fundamental and widely used operation in image processing that affects how light or dark and the intensity of colors appear in an image.

Conceptual Overview

- **Brightness adjustment** refers to shifting the overall intensity of pixels in an image. Increasing brightness makes the image lighter by adding a constant value to all pixel intensities, while decreasing brightness makes it darker by subtracting a constant value.
- **Contrast adjustment** changes the difference in luminance or color between pixels. Increasing contrast makes the darks darker and lights lighter, enhancing the details and edges. Decreasing contrast brings pixel values closer together, producing a flatter, duller image.

Mathematical Model

In OpenCV and many image processing frameworks, brightness and contrast are adjusted via a linear transformation for each pixel value II:

Inew=α×Ioriginal+βInew=α×Ioriginal+β

Where:

- αα (alpha) controls **contrast**. If α>1α>1, contrast increases; if 0<α<10<α<1, contrast decreases.
- ββ (beta) controls **brightness**. Positive ββ increases brightness, negative ββ decreases it.

Effects on Image

- With **increased contrast (α>1α>1)**, the difference between bright and dark areas becomes more pronounced, helping highlight textures and features.
- With **decreased contrast (0<α<10<α<1)**, the image becomes washed-out or faded.
- Adjusting **brightness (ββ)** shifts the entire image luminance up or down without altering the relative contrast.

Considerations for Advanced Processing

- **Pixel Value Clipping:** Since pixel values have fixed ranges (e.g., 0-255 for 8-bit images), values after the transformation might be clipped to the valid range, which can affect image quality.
- **Color Spaces:** Sometimes brightness and contrast adjustments are better performed in different color spaces (e.g., HSV or LAB), where luminance is separated from color information, to avoid color distortions.
- **Gamma Correction:** Adjusting brightness and contrast non-linearly via gamma correction may offer more visually pleasing results for certain applications.
- **Histograms:** Contrast is also associated with the spread of the histogram of pixel intensities; histogram equalization or stretching are more advanced contrast enhancement techniques.

In summary, **OpenCV's brightness and contrast adjustment is typically achieved by a simple linear scaling and shifting of pixel intensities** controlled by parameters αα and ββ, which allows flexible enhancement or reduction of image brightness and contrast suitable for many advanced image processing tasks.

Adding Text to Images

OpenCV is a versatile library widely used for image processing and computer vision tasks. One common advanced operation is adding text annotations directly onto images, which is essential for tasks like watermarking, labeling, debugging, or creating visually informative images.

Adding Text to Images with OpenCV

OpenCV provides a built-in function cv2.putText() to overlay text on images. This function is highly customizable and supports several font types, sizes, colors, thickness, and line types.

Basic Syntax of cv2.putText()

```
cv2.putText(img,            # Image to draw text on
        text,           # Text string to write
        org,            # Bottom-left corner of the text string (x, y)
        fontFace,       # Font type (enum)
        fontScale,      # Font scale (float)
        color,          # Text color (B, G, R)
        thickness=1,    # Thickness of the text strokes
        lineType=cv2.LINE_AA)  # Line type (e.g. anti-aliased)
```

Example: Adding Text to an Image

```
import cv2

import numpy as np


# Create a blank white image

img = np.ones((400, 600, 3), dtype=np.uint8) * 255


# Text parameters

text = "OpenCV Text Example"

position = (50, 200)  # x=50, y=200

font = cv2.FONT_HERSHEY_COMPLEX

font_scale = 2

color = (0, 0, 255)  # Red color in BGR

thickness = 3


# Put text on the image

cv2.putText(img, text, position, font, font_scale, color, thickness, cv2.LINE_AA)


# Display the image

cv2.imshow("Image with Text", img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Tips for Advanced Text Addition

1. **Font Selection**: OpenCV supports several fonts like FONT_HERSHEY_SIMPLEX, FONT_HERSHEY_COMPLEX, FONT_HERSHEY_SCRIPT_SIMPLEX, etc. Choose fonts that fit the context.
2. **Text Size/Scaling**: Adjust fontScale to resize the text appropriately.

3. **Color and Thickness**: Use contrasting colors for visibility, and adjust thickness for bold or thin text.
4. **Positioning**: The org parameter gives the bottom-left coordinate for text.
   Use cv2.getTextSize() to measure your text size for dynamic placement.
5. **Anti-Aliasing**: Using cv2.LINE_AA helps produce smoother text.
6. **Multi-line Text**: OpenCV doesn't have built-in support for multiline text—split your text by and draw each line separately by adjusting the y-coordinate.
7. **Transparency / Overlay**: For watermark purposes, you might want to blend the text with the image using alpha blending to make it semi-transparent.

Measuring Text Size Example

(text_width, text_height), baseline = cv2.getTextSize(text, font, font_scale, thickness)

print(f"Text size: {text_width} x {text_height}, baseline: {baseline}")

This helps in aligning or centering text within an image.

Summary

- Use cv2.putText() for adding text.
- Customize font, size, color, thickness.
- Use cv2.getTextSize() to get text dimension.
- For multiline, manually handle line breaks.
- Consider anti-aliasing for better visual quality.
- For transparency effects, combine text on a separate layer and blend with the original image.

OpenCV's text rendering, while simple, is powerful enough for most annotation needs while performing other advanced image processing tasks.

If you want sample code for adding transparent text or multiline support, feel free to ask!

Adding Text to Images

Adding text to images is an important feature in advanced image processing with OpenCV, often used for annotations, labeling, watermarking, or creating customized visuals.

Key Concepts for Adding Text to Images in OpenCV:

- **Functionality**: OpenCV provides built-in support for drawing text on images using functions that specify the content, font style, size, color, and position on the image. This helps in visually conveying information directly on the image.
- **Font Types**: Various fonts are available in OpenCV such as FONT_HERSHEY_SIMPLEX, FONT_HERSHEY_COMPLEX, and others that can be chosen depending on the desired style and clarity.
- **Positioning**: Text placement on the image is controlled by specifying the bottom-left corner coordinates where the text will start. This allows precise control over where the text appears.
- **Font Scale and Thickness**: The size of the text is adjustable through the font scale parameter, while the thickness parameter controls the boldness or stroke width of the text. This ensures readability and visual impact depending on the image size and resolution.
- **Color Space**: Text color is defined in BGR (Blue, Green, Red) format, consistent with OpenCV's color conventions, allowing any desired color to be used based on the image context.
- **Transparency and Overlay**: While adding plain text is straightforward, advanced use cases may include controlling the transparency (opacity) of the text overlay or blending it smoothly with the image background for better aesthetics.
- **Applications**: Adding text is frequently used for creating watermarks, annotating images with diagnostic information in medical imaging, marking objects in object detection tasks, and increasing image interpretability in machine vision and multimedia projects.

Smoothing Images: Median Filter

Median filtering is a popular nonlinear technique used in image processing to reduce noise while preserving edges and details better than standard linear smoothing filters.

What is Median Filtering?

Median filtering replaces each pixel's value in an image with the median value of the pixels within a defined neighborhood (usually a square window, like 3x3 or 5x5) around that pixel. Unlike averaging filters, which calculate the mean of the neighborhood pixels, the median filter selects the middle value when the neighboring pixel values are sorted in order.

How Median Filtering Works

1. For each pixel in the image, consider the pixel values of its neighboring pixels within the filter window.
2. Sort these pixel values.

3. Replace the original pixel's value with the median (middle) value from the sorted list.
4. Move to the next pixel and repeat the process over the entire image.

Advantages of Median Filtering

- **Effective at Removing Salt-and-Pepper Noise**: Median filters are especially good at removing impulse noise, often called salt-and-pepper noise, without blurring sharp edges.
- **Edge Preservation**: Unlike averaging filters that smooth the image uniformly (blurring edges), median filtering preserves edges and sharp boundaries.
- **Nonlinear Filter**: Median filtering is nonlinear, so it can handle noise that linear filters cannot efficiently remove.

Applications

- Noise reduction in digital photographs and scanned images.
- Preprocessing step for edge detection and segmentation.
- Medical imaging to reduce artifacts while maintaining structural details.
- Enhancing images from surveillance and remote sensing.

Limitations

- Median filtering can distort fine details in images, especially if too large a window size is chosen.
- It requires sorting operations for each pixel, which can be computationally more expensive compared to linear filters.

Bilateral Filter

Definition

The **Bilateral Filter** is an edge-preserving and noise-reducing smoothing filter extensively used in image processing. Unlike linear smoothing filters (e.g., Gaussian blur), it **smooths images while preserving edges** by considering both spatial proximity and intensity similarity in the filtering process.

Core Principle

- It combines **domain filtering** (spatial closeness) and **range filtering** (photometric similarity) to weight neighboring pixels.
- The output pixel is a weighted average of nearby pixels where the weights decrease both with distance and difference in intensity.

Mathematically, for a pixel pp, the filtered value Ifiltered(p)Ifiltered(p) is computed as:

Ifiltered(p)=1Wp∑q∈ΩI(q)·fs(||p−q||)·fr(|I(p)−I(q)|)Ifiltered(p)=Wp1q∈Ω∑I(q)·fs(||p−q||)·fr(|I(p)−I(q)|)

where:

- I(q)I(q) is the intensity at pixel qq,
- ΩΩ is the neighborhood or local window around pp,
- fsfs is the spatial Gaussian weighting function (distance-based),
- frfr is the range Gaussian weighting function (intensity difference-based),
- Wp=∑q∈Ωfs(||p−q||)·fr(|I(p)−I(q)|)Wp=∑q∈Ωfs(||p−q||)·fr(|I(p)−I(q)|) is the normalization factor.

Advantages

- **Edge preservation:** respects edges by reducing smoothing across intensity boundaries.
- **Noise reduction:** removes noise in smooth regions effectively.

Applications

- Image denoising,
- HDR tone mapping,
- Texture removal,
- Depth and normal map smoothing in computer graphics.

Computational Considerations

- Bilateral filtering is **computationally intensive** (non-linear and non-separable).
- Various approximations and acceleration schemes have been developed (e.g., using quantized range or using bilateral grid methods).

Changing the Shape of Images

AI tools for image smoothing and reshaping harness advanced machine learning algorithms to enhance image quality by reducing noise, smoothing textures, and even transforming the shape or content of images in creative or corrective ways. These technologies enable users across skill levels—from professionals to hobbyists—to improve or alter images quickly without extensive manual editing.

How AI Image Smoothing Works

- **Noise Reduction:** AI models identify grain, noise, and pixelation often caused by low light or poor capture settings, then intelligently reduce these artifacts while preserving important details.
- **Detail Preservation:** Unlike simple blurring, AI smoothing techniques selectively smooth unwanted roughness and imperfections while maintaining edges and textures critical for clarity.
- **Clarity Enhancement:** By analyzing image regions, AI optimizes brightness, contrast, and sharpness to improve overall crispness.

Popular Online and Offline AI Image Smoothers

1. **Pokecut AI Image Smoother**: An easy online tool for removing noise/grain and enhancing image clarity without registration. Ideal for portraits or landscapes. Free for limited daily use.
2. **Remaker AI Photo Enhancer**: Offers noise reduction, detail restoration, color correction, resolution upscaling, and background enhancement. No sign-up required and suitable for various image types.
3. **Arting AI Image Enhancer**: Online free tool to upscale and sharpen images up to 4X resolution for digital or print use. Supports quick edits and high-quality exports.

AI Tools for Changing Image Shape or Content

- **OpenArt AI Photo Editor:** Allows users to select image areas and describe desired modifications (e.g., changing shapes, adding objects) through natural language commands for intuitive reshaping or retouching.
- **Fotor AI Photo Editor:** Comprehensive AI suite featuring background removal, object removal, image extending (outpainting), face retouching, and shape adjustments—all with minimal user inputs or via text prompts.
- **Media.io AI Photo Retoucher:** Specializes in portrait beautification, skin smoothing, blemish removal, and background changes, enabling professional-level touch-ups with ease.

Use Cases for Smoothing and Shape Changing AI Tools

- **Photography:** Improve image quality captured in challenging environments; remove noise; sharpen subjects while retaining natural details.
- **E-commerce/Product Photos:** Smooth and reshape product images for enhanced visual appeal; remove distractions; adjust shapes for better presentation.

- **Social Media/Content Creation:** Create polished portraits and unique visuals by smoothing skin, removing blemishes, or reshaping via AI-assisted retouching.
- **Graphic Design:** Modify shapes or artistic elements without manual redraw; add or remove items; extend canvases intelligently for creative effects.

Advantages of AI-Based Image Smoothing and Shape Alteration

- **Speed and Ease:** One-click enhancements with immediate visible results, no advanced skills required.
- **Consistency:** Automated adjustments maintain uniform quality across batches of photos.
- **Cost-effective:** Many tools provide free tiers or affordable subscriptions, removing the need for expensive manual editing.
- **Versatility:** Suitable for portraits, landscapes, product shots, artwork, and more.
- **Accessibility:** Many online tools work directly in browsers or mobile apps, requiring no installation.

Recommendations

1. Evaluate your needs: smoothing noisy images requires noise reduction-focused AI smoothers (e.g., Pokecut, Remaker), while reshaping images benefits from flexible AI editors supporting object removal and background changes (e.g., Fotor, OpenArt).
2. Use free online tools for quick fixes or experimentation; consider paid versions or desktop apps for professional-grade work.
3. Always review results to avoid over-smoothing that can produce unnatural visuals. Many tools allow adjustment of enhancement intensity.

Effecting Image Thresholding

Image thresholding is an important technique in image processing that helps to clearly separate objects from the background, making images simpler to analyze and useful in many practical applications like medical imaging, document scanning, and object detection.

What is Image Thresholding?

Image thresholding is a method used to convert a grayscale image (with many shades of gray) into a simple black-and-white (binary) image. This is done by choosing a specific **threshold value**:

- Pixels with intensity above the threshold become white (foreground)
- Pixels below the threshold become black (background)

This simplifies the image by separating important parts (objects) from the background, making it easier for computers or humans to recognize and process the objects.

Why is Thresholding Important?

- **Simplifies complex images:** It reduces many shades of gray to just two colors, making it easier to understand the image.
- **Enhances object visibility:** Objects of interest become clearer and easier to detect.
- **Reduces noise:** Helps remove unnecessary details that can distract from the main objects.
- **Used in many real-life applications:** Such as medical scans to spot tumors, document scanning to recognize text (for OCR), quality inspection in factories, and even self-driving cars to recognize road signs or obstacles.

Types of Thresholding Techniques (Simple Explanation)

- **Global Thresholding:** Uses one fixed threshold value for the whole image. Best when lighting is uniform.
- **Local (Adaptive) Thresholding:** Uses different threshold values for different parts of the image. Useful when the lighting varies.
- **Otsu's Method:** Automatically calculates the best threshold based on the image's pixel values.

Example in Real Life

Imagine scanning a handwritten note. Thresholding helps convert the note into a clear black-and-white image where the words stand out cleanly from the paper background so that a computer can read the text correctly.

Calculating Gradients

1. What is an Image Gradient?

- The image gradient is a vector representing the change in intensity or color in an image.
- It measures how pixel values change in horizontal (x) and vertical (y) directions.
- At each pixel, the gradient points in the direction of the greatest increase in intensity.
- The magnitude (strength) and orientation (direction) of the gradient are used to detect edges, boundaries, and features.

2. Mathematical Definition

For an image represented as a function I(x,y)I(x,y), the gradient is:

ablaI=[∂I∂x,∂I∂y]ablaI=[∂x∂I,∂y∂I]

- ∂I∂x∂x∂I: Change in intensity in the horizontal direction.
- ∂I∂y∂y∂I: Change in intensity in the vertical direction.

The gradient magnitude GG and orientation θθ are given by:

G=(∂I∂x)2+(∂I∂y)2G=(∂x∂I)2+(∂y∂I)2

θ=arctan⁡2(∂I∂y,∂I∂x)θ=arctan2(∂y∂I,∂x∂I)

3. Computing Gradients Manually

- Using central difference approximation for partial derivatives:

∂I∂x≈I(x+1,y)−I(x−1,y)∂x∂I≈I(x+1,y)−I(x−1,y)

∂I∂y≈I(x,y+1)−I(x,y−1)∂y∂I≈I(x,y+1)−I(x,y−1)

- This uses pixel intensity differences between neighboring pixels horizontally and vertically.

4. Gradient Filters (Kernels)

- For efficient and robust gradient calculation, convolution kernels/filters are applied over the image.
- Common gradient filters:
    - **Sobel Filter**: Uses 3x3 kernels with weighted central pixels, balancing smoothing and differentiation.
    - **Prewitt Filter**: Simpler 3x3 kernels, sensitive to vertical and horizontal edges.
    - **Roberts Filter**: Uses 2x2 kernels focusing on diagonal differences.
    - **Scharr Filter**: Improved Sobel with better rotational symmetry; more accurate gradient approximation.
- These filters compute GxGx and GyGy, approximating horizontal and vertical derivatives.

5. Practical Considerations

- **Noise Reduction:** Smoothing (e.g., Gaussian blur) before gradient calculation reduces noise-induced errors.
- **Data Types:** Gradients can be positive or negative; keeping intermediate computations in signed or float types (e.g., cv.CV_16S, cv.CV_64F) preserves edge polarity.

- **Edge Detection:** High gradient magnitudes indicate edges. Gradient orientation helps determine edge directions.
- **Implementation:** Libraries like OpenCV provide functions (cv2.Sobel(), cv2.Scharr(), cv2.Laplacian()) to compute gradients efficiently.

6. Workflow for Edge Detection Using Gradients:

1. Convert image to grayscale.
2. Apply smoothing to reduce noise.
3. Calculate gradients GxGx and GyGy using filters.
4. Compute gradient magnitude and orientation from GxGx and GyGy.
5. Threshold or use additional methods (e.g., non-maximum suppression) to find edges.

7. Applications of Image Gradients

- Edge detection (Canny, Sobel edge detectors).
- Feature extraction (SIFT, HOG).
- Image segmentation.
- Texture analysis.
- Object recognition and tracking.
- Robotics and computer vision for path detection, surface recognition, etc.

Performing Histogram Equalization

Histogram Equalization is a widely used technique in image processing aimed at improving the contrast of an image by effectively spreading out the most frequent intensity values.

Step-by-Step Overview of Histogram Equalization

1. **Goal**
   Enhance the global contrast of the image, especially when the original image's intensity values are limited to a narrow range, making features indistinct.
2. **Input**
   A grayscale image of dimensions M×NM×N, where each pixel has an intensity value rkrk in the range [0,L−1][0,L−1], often L=256L=256.
3. **Procedure**

- **Calculate the Histogram**:
  Compute the frequency nknk of each gray level rkrk.

nk=number of pixels with intensity rknk=number of pixels with intensity rk

- **Normalize the Histogram (PDF)**:
  Calculate the probability distribution function (PDF) for each gray level:

pr(rk)=nkM×Npr(rk)=M×Nnk

- **Compute Cumulative Distribution Function (CDF)**:
  The CDF gives the cumulative probability up to gray level rkrk:

cdf(rk)=∑j=0kpr(rj)cdf(rk)=j=0∑kpr(rj)

- **Define the Transformation Function**:
  Each input pixel value rkrk is mapped to a new intensity sksk by the function:

sk=(L−1)×cdf(rk)sk=(L−1)×cdf(rk)

The function is monotonic and maps the intensity levels to span the entire output range.

- **Map Original Intensities to Equalized Values**:
  Replace each pixel intensity rkrk in the original image with its corresponding equalized value sksk from the transformation.

4. **Output**
   An image with enhanced contrast where intensity values are more evenly distributed across the available range.

Intuition and Effects

- Histogram equalization stretches out clusters of intensity values and compresses sparsely occurring ones, thus increasing the overall contrast.
- It tends to turn a narrow, peaked histogram into a wider and more uniform histogram.
- This often makes features more visible, especially in images that are dark or washed out.

Extensions and Variants

- **Adaptive Histogram Equalization (AHE):**
  Histogram equalization applied locally to small regions instead of the entire image, useful for images with varying lighting.
- **Contrast Limited Adaptive Histogram Equalization (CLAHE):**
  Further modifies AHE by limiting the contrast amplification to reduce noise amplification.

Mathematical Summary

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j) s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j)$$

where $s_k s_k$ is the new pixel intensity, $r_k r_k$ the original pixel intensity, and $p_r p_r$ the normalized histogram.